

Cours Théorique Complet

Séance 3 : Installation & Premiers Composants React

M203 - Développement Front-End avec React

ISTA - 2ème Année Développement Digital

2025/2026

Contents

1	Introduction	2
1.1	Contexte Pédagogique	2
1.2	Objectifs de la Séance	2
1.3	Prérequis Validés	2
2	Installation de l'Environnement React	3
2.1	Node.js et npm : Fondations du Développement Moderne	3
2.1.1	Qu'est-ce que Node.js ?	3
2.1.2	npm : Le Gestionnaire de Packages	3
2.2	Vite : L'Outil de Build Moderne	4
2.2.1	Introduction à Vite	4
2.2.2	Pourquoi Vite est Recommandé en 2025	4
2.2.3	Installation d'un Projet avec Vite	4
2.3	Structure d'un Projet React avec Vite	5
2.3.1	Arborescence Complète	5
2.3.2	Fichiers Clés à Comprendre	5
3	Les Composants React	8
3.1	Concept Fondamental	8
3.2	Anatomie d'un Composant	8
3.2.1	Structure de Base	8
3.2.2	Règles Impératives	8
3.3	JSX : JavaScript XML	10
3.3.1	Définition	10
3.3.2	Transformation JSX → JavaScript	10
3.3.3	Règles du JSX	10
3.4	Export et Import de Composants	12
3.4.1	Export Default	12
3.4.2	Export Nommé (Named Export)	12
3.4.3	Chemins d'Import	12
4	Créer un Premier Composant	14
4.1	Exemple Guidé : Composant Header	14
4.1.1	Étape 1 : Créer le Fichier	14
4.1.2	Étape 2 : Écrire le Composant	14
4.1.3	Étape 3 : Créer les Styles	14

4.1.4	Étape 4 : Utiliser dans App.jsx	15
4.2	Exemple : Composant Card	15
5	Composition de Composants	17
5.1	Principe de Composition	17
5.2	Exemple de Composition	17
6	Bonnes Pratiques	19
6.1	Organisation des Fichiers	19
6.2	Conventions de Nommage	19
6.3	Commentaires en JSX	19
7	Debugging et DevTools	20
7.1	Console du Navigateur	20
7.2	React Developer Tools	20
7.3	console.log() en React	20
8	Récapitulatif	21
8.1	Concepts Clés Maîtrisés	21
8.2	Prochaines Étapes (Séance 4)	21
9	Ressources Professionnelles	22
9.1	Documentation Officielle	22
9.2	Tutoriels et Guides	22
9.3	Vidéos (Recommandées)	22
9.4	Pratique Interactive	22
9.5	Communauté et Support	23
9.6	Livres Recommandés	23

1 Introduction

1.1 Contexte Pédagogique

Après avoir découvert **pourquoi utiliser React** (Séance 1) et avoir maîtrisé les **concepts JavaScript ES6** nécessaires (Séance 2), nous sommes maintenant prêts à installer un environnement de développement React et à créer nos premiers composants.

Cette séance marque le début de la pratique concrète : nous allons transformer les concepts théoriques en code fonctionnel.

1.2 Objectifs de la Séance

À la fin de cette séance, vous serez capable de :

1. Installer et configurer un environnement de développement React (Vite)
2. Comprendre la structure d'un projet React
3. Créer des composants React fonctionnels
4. Maîtriser les règles du JSX
5. Exporter et importer des composants
6. Composer une page avec plusieurs composants

1.3 Prérequis Validés

- Concepts JavaScript ES6 (let/const, arrow functions, destructuring, spread, map/filter)
- Compréhension du concept de SPA (Single Page Application)
- Différence entre JavaScript Vanilla et React

2 Installation de l'Environnement React

2.1 Node.js et npm : Fondations du Développement Moderne

2.1.1 Qu'est-ce que Node.js ?

Node.js

Node.js est un environnement d'exécution JavaScript qui permet d'exécuter du code JavaScript en dehors du navigateur, directement sur votre machine.

Source : Node.js Official Documentation - <https://nodejs.org/en/about>

Pourquoi Node.js pour React ?

Bien que React s'exécute dans le navigateur, nous avons besoin de Node.js pour :

1. **npm (Node Package Manager)** : Gérer les dépendances (bibliothèques)
2. **Serveur de développement** : Hot reload, compilation JSX
3. **Outils de build** : Transpilation, minification, optimisation
4. **Scripts de développement** : Automatiser les tâches

Référence : W3Schools Node.js Tutorial - <https://www.w3schools.com/nodejs/>

2.1.2 npm : Le Gestionnaire de Packages

npm (Node Package Manager)

npm est le gestionnaire de packages par défaut pour Node.js. Il permet d'installer, partager et gérer des bibliothèques JavaScript.

Source : npm Documentation - <https://docs.npmjs.com/about-npm>

Commandes npm essentielles :

```
# Verifier la version
npm --version

# Installer les dependances du projet
npm install

# Installer un package specifique
npm install nom-du-package

# Lancer un script defini dans package.json
npm run nom-du-script
```

Référence : GeeksforGeeks npm Tutorial - <https://www.geeksforgeeks.org/node-js-npm-node-packa>

2.2 Vite : L'Outil de Build Moderne

2.2.1 Introduction à Vite

Vite

Vite (prononcé "vite" en français, signifie "rapide") est un outil de build nouvelle génération créé par Evan You (créateur de Vue.js) qui offre une expérience de développement ultra-rapide.

Source : Vite Official Documentation - <https://vitejs.dev/guide/why.html>

2.2.2 Pourquoi Vite est Recommandé en 2025

1. Performance Exceptionnelle

Métrique	Vite	Create React App
Démarrage serveur	1-2 sec	10-20 sec
Hot Module Replacement	Instantané	2-3 sec
Taille bundle dev	Léger	Lourd
Premier chargement	Rapide	Moyen

Source : Vite Performance Benchmarks - <https://vitejs.dev/guide/why.html#slow-server-start>

2. Architecture Moderne

Vite utilise les **ES modules natifs du navigateur** pendant le développement, ce qui élimine le besoin de bundling et accélère considérablement le rafraîchissement.

3. Recommandation Officielle React

Depuis 2023, la documentation officielle React recommande d'utiliser des frameworks modernes comme Vite au lieu de Create React App.

Source : React Official Documentation - <https://react.dev/learn/start-a-new-react-project>

2.2.3 Installation d'un Projet avec Vite

Commande d'installation :

```
# Créer un nouveau projet React avec Vite
npm create vite@latest mon-projet-react -- --template react

# Naviguer dans le dossier
cd mon-projet-react

# Installer les dépendances
npm install

# Lancer le serveur de développement
npm run dev
```

Explication de la commande :

- `npm create vite@latest` : Utilise l'outil de création Vite
- `mon-projet-react` : Nom du projet (personnalisable)
- `-- --template react` : Utilise le template React pré-configuré

Référence : Vite Getting Started - <https://vitejs.dev/guide/>

2.3 Structure d'un Projet React avec Vite

2.3.1 Arborescence Complète

```
mon-projet-react/  
  node_modules/          # Dépendances (ne pas modifier)  
  public/                 # Fichiers statiques  
    vite.svg  
  src/                    # CODE SOURCE (ici qu'on travaille)  
    assets/              # Images, fonts, etc.  
    App.css              # Styles du composant App  
    App.jsx              # Composant principal  
    index.css            # Styles globaux  
    main.jsx             # Point d'entrée de l'application  
  .gitignore              # Fichiers à ignorer par Git  
  index.html              # Page HTML unique (SPA)  
  package.json            # Configuration du projet  
  package-lock.json       # Versions exactes des dépendances  
  vite.config.js          # Configuration Vite  
  README.md               # Documentation du projet
```

2.3.2 Fichiers Clés à Comprendre

1. package.json - Le Manifeste du Projet

```
1 {  
2   "name": "mon-projet-react",  
3   "private": true,  
4   "version": "0.0.0",  
5   "type": "module",  
6   "scripts": {  
7     "dev": "vite",           // Lancer serveur dev  
8     "build": "vite build",   // Build production  
9     "preview": "vite preview" // Previsualiser build  
10  },  
11  "dependencies": {  
12    "react": "^18.3.1",      // React core  
13    "react-dom": "^18.3.1"   // React pour DOM  
14  },  
15  "devDependencies": {  
16    "@vitejs/plugin-react": "^4.2.1",  
17    "vite": "^5.0.8"  
18  }  
19 }
```

Points importants :

- **scripts** : Commandes npm disponibles
- **dependencies** : Bibliothèques nécessaires en production
- **devDependencies** : Outils de développement uniquement

Référence : npm package.json Guide - <https://docs.npmjs.com/cli/v10/configuring-npm/package-json>

2. index.html - La Page Unique

```

<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Mon Projet React</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>

```

Points clés :

- `<div id="root"></div>` : Point d'ancrage de React
- `<script type="module" src="/src/main.jsx">` : Charge l'application React

3. src/main.jsx - Point d'Entrée React

```

1 import React from 'react'
2 import ReactDOM from 'react-dom/client'
3 import App from './App.jsx'
4 import './index.css'
5
6 ReactDOM.createRoot(document.getElementById('root')).render(
7   <React.StrictMode>
8     <App />
9   </React.StrictMode>,
10 )

```

Explication ligne par ligne :

1. `import React from 'react'` : Importe la bibliothèque React
2. `import ReactDOM from 'react-dom/client'` : Importe le moteur de rendu DOM
3. `import App from './App.jsx'` : Importe le composant principal
4. `ReactDOM.createRoot(...)` : Crée la racine React dans le `div#root`
5. `<React.StrictMode>` : Active les vérifications supplémentaires en développement
6. `<App />` : Rend le composant App

Source : React createRoot API - <https://react.dev/reference/react-dom/client/createRoot>

4. src/App.jsx - Composant Principal

```

1 import { useState } from 'react'
2 import './App.css'
3
4 function App() {
5   const [count, setCount] = useState(0)

```

```
6
7   return (
8     <div className="App">
9       <h1>Mon Application React</h1>
10      <button onClick={() => setCount(count + 1)}>
11        count is {count}
12      </button>
13    </div>
14  )
15 }
16
17 export default App
```

Structure d'un composant :

1. Imports (dépendances, styles)
2. Déclaration de la fonction
3. Logique du composant (state, fonctions)
4. Return (JSX)
5. Export

3 Les Composants React

3.1 Concept Fondamental

Composant React

Un **composant** est une fonction JavaScript qui retourne du JSX (code HTML-like). C'est un bloc de construction réutilisable de l'interface utilisateur.

Source : React Components and Props - <https://react.dev/learn/your-first-component>

3.2 Anatomie d'un Composant

3.2.1 Structure de Base

```
1 // 1. Imports
2 import './MonComposant.css'
3
4 // 2. Fonction composant (MAJUSCULE obligatoire)
5 function MonComposant() {
6   // 3. Logique (variables, fonctions, state)
7   const titre = "Bonjour React";
8
9   // 4. Return (JSX)
10  return (
11    <div className="mon-composant">
12      <h1>{titre}</h1>
13      <p>Mon premier composant</p>
14    </div>
15  );
16 }
17
18 // 5. Export
19 export default MonComposant;
```

3.2.2 Règles Impératives

1. Nom en PascalCase

```
1 //      CORRECT
2 function Header() { }
3 function CardProduit() { }
4 function UserProfile() { }
5
6 //      INCORRECT
7 function header() { } // Minuscule
8 function card_produit() { } // Snake case
```

Pourquoi ? React différencie les composants (PascalCase) des éléments HTML natifs (minuscules).

Source : React Naming Conventions - <https://react.dev/learn/your-first-component#components-all-the-way-down>

2. Retourner du JSX

Chaque composant DOIT retourner du JSX (ou null).

```

1 //      CORRECT
2 function Composant() {
3     return <div>Contenu</div>;
4 }
5
6 //      INCORRECT
7 function Composant() {
8     console.log("Pas de return"); // undefined
9 }

```

3. Une Seule Racine (Root Element)

```

1 //      INCORRECT - Plusieurs racines
2 function App() {
3     return (
4         <h1>Titre</h1>
5         <p>Paragraphe</p>
6     );
7 }
8
9 //      CORRECT - Une div parent
10 function App() {
11     return (
12         <div>
13             <h1>Titre</h1>
14             <p>Paragraphe</p>
15         </div>
16     );
17 }
18
19 //      CORRECT - Fragment React (syntaxe courte)
20 function App() {
21     return (
22         <>
23             <h1>Titre</h1>
24             <p>Paragraphe</p>
25         </>
26     );
27 }

```

Source : React Fragments - <https://react.dev/reference/react/Fragment>

3.3 JSX : JavaScript XML

3.3.1 Définition

JSX (JavaScript XML)

JSX est une extension de syntaxe pour JavaScript qui permet d'écrire du code HTML-like directement dans du JavaScript. Il est ensuite transformé en appels de fonctions JavaScript par Babel.

Source : React JSX Introduction - <https://react.dev/learn/writing-markup-with-jsx>

3.3.2 Transformation JSX → JavaScript

Code JSX :

```
1 const element = <h1 className="titre">Bonjour</h1>;
```

Transformé en JavaScript :

```
1 const element = React.createElement(  
2   'h1',  
3   { className: 'titre' },  
4   'Bonjour'  
5 );
```

Référence : Babel JSX Transform - <https://babeljs.io/docs/babel-plugin-transform-react-jsx>

3.3.3 Règles du JSX

Règle 1 : className au lieu de class

```
1 // INCORRECT  
2 <div class="container">Contenu</div>  
3  
4 // CORRECT  
5 <div className="container">Contenu</div>
```

Pourquoi ? class est un mot-clé réservé en JavaScript (pour les classes ES6).

Source : W3Schools React JSX - https://www.w3schools.com/react/react_jsx.asp

Règle 2 : Fermer toutes les balises

```
1 // INCORRECT (HTML)  
2   
3 <br>  
4 <input type="text">  
5  
6 // CORRECT (JSX)  
7   
8 <br />  
9 <input type="text" />
```

Règle 3 : Interpolation avec { }

```
1 function Greeting() {  
2   const nom = "Ahmed";  
3   const age = 20;
```

```

4
5     return (
6         <div>
7             <p>Nom : {nom}</p>
8             <p>Age : {age}</p>
9             <p>Calcul : {5 + 3}</p>
10            <p>Condition : {age >= 18 ? "Majeur" : "Mineur"}</p>
11        </div>
12    );
13 }

```

Vous pouvez mettre dans { } :

- Variables : {nom}
- Expressions : {5 + 3}
- Fonctions : {Math.random()}
- Ternaires : {condition ? "oui" : "non"}
- Méthodes : {array.map(...)}

Source : React JavaScript in JSX - <https://react.dev/learn/javascript-in-jsx-with-curly-braces>

Règle 4 : camelCase pour les attributs

```

1 // HTML natif
2 <button onclick="handleClick()">Cliquez</button>
3
4 // JSX React
5 <button onClick={handleClick}>Cliquez</button>

```

Attributs courants :

- onclick → onClick
- onchange → onChange
- onsubmit → onSubmit
- for → htmlFor
- tabIndex → tabIndex

Référence : GeeksforGeeks React JSX - <https://www.geeksforgeeks.org/reactjs-jsx-introduction/>

3.4 Export et Import de Composants

3.4.1 Export Default

Fichier : Header.jsx

```
1 function Header() {
2     return <h1>Mon Header</h1>;
3 }
4
5 export default Header;
```

Import :

```
1 import Header from './Header';
2
3 // Utilisation
4 <Header />
```

Caractéristiques :

- Un seul export default par fichier
- Peut être importé sous n'importe quel nom
- Syntaxe recommandée pour les composants React

3.4.2 Export Nommé (Named Export)

Fichier : utils.js

```
1 export function formatDate(date) {
2     return date.toLocaleDateString();
3 }
4
5 export function capitalize(str) {
6     return str.charAt(0).toUpperCase() + str.slice(1);
7 }
```

Import :

```
1 import { formatDate, capitalize } from './utils';
2
3 // Utilisation
4 const date = formatDate(new Date());
```

Référence : MDN ES6 Modules - <https://developer.mozilla.org/fr/docs/Web/JavaScript/Guide/Modules>

3.4.3 Chemins d'Import

```
1 // Import depuis le meme dossier
2 import Header from './Header'
3
4 // Import depuis un sous-dossier
5 import Card from './components/Card'
6
7 // Import depuis un dossier parent
```

```
8 import utils from '../utils'
9
10 // Import depuis node_modules
11 import React from 'react'
```

4 Créer un Premier Composant

4.1 Exemple Guidé : Composant Header

4.1.1 Étape 1 : Créer le Fichier

Créer `src/components/Header.jsx`

4.1.2 Étape 2 : Écrire le Composant

```
1 import './Header.css'
2
3 function Header() {
4   return (
5     <header className="header">
6       <div className="logo">
7         <h1>Mon Site React</h1>
8       </div>
9       <nav className="nav">
10        <a href="#home">Accueil</a>
11        <a href="#about">  propos</a>
12        <a href="#services">Services</a>
13        <a href="#contact">Contact</a>
14      </nav>
15    </header>
16  );
17 }
18
19 export default Header;
```

4.1.3 Étape 3 : Créer les Styles

Créer `src/components/Header.css`

```
.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px 40px;
  background: #282c34;
  color: white;
}

.logo h1 {
  margin: 0;
  font-size: 24px;
}

.nav {
  display: flex;
  gap: 30px;
}
```

```

.nav a {
  color: white;
  text-decoration: none;
  font-size: 16px;
  transition: color 0.3s;
}

.nav a:hover {
  color: #61dafb;
}

```

4.1.4 Étape 4 : Utiliser dans App.jsx

```

1 import Header from './components/Header'
2 import './App.css'
3
4 function App() {
5   return (
6     <div className="App">
7       <Header />
8       <main>
9         <p>Contenu principal</p>
10      </main>
11    </div>
12  );
13 }
14
15 export default App;

```

4.2 Exemple : Composant Card

```

1 import './Card.css'
2
3 function Card() {
4   return (
5     <div className="card">
6       
10      <div className="card-content">
11        <h3>Nom du Produit</h3>
12        <p className="description">
13          Description courte du produit
14        </p>
15        <div className="card-footer">
16          <span className="price">299 DH</span>
17          <button className="btn-add">
18            Ajouter au panier
19          </button>
20        </div>
21      </div>

```



```
22         </div>
23     );
24 }
25
26 export default Card;
```

5 Composition de Composants

5.1 Principe de Composition

La puissance de React réside dans la capacité de **composer des composants** - assembler des petits composants pour créer des interfaces complexes.

Composition

En React, on construit des interfaces complexes en **assemblant** de nombreux petits composants, comme des briques LEGO.

Source : React Composition vs Inheritance - <https://react.dev/learn/thinking-in-react>

5.2 Exemple de Composition

```
1 import Header from './components/Header'
2 import Card from './components/Card'
3 import Footer from './components/Footer'
4 import './App.css'
5
6 function App() {
7   return (
8     <div className="App">
9       {/* Header component */}
10      <Header />
11
12      {/* Main content */}
13      <main className="main-content">
14        <h2>Nos Produits</h2>
15
16        <div className="cards-container">
17          {/* Multiple Card components */}
18          <Card />
19          <Card />
20          <Card />
21        </div>
22      </main>
23
24      {/* Footer component */}
25      <Footer />
26    </div>
27  );
28 }
29
30 export default App;
```

Avantages de la composition :

1. **Réutilisabilité** : Un composant = utilisable partout
2. **Maintenabilité** : Modifier un composant = mise à jour partout
3. **Lisibilité** : Code organisé et structuré
4. **Testabilité** : Tester chaque composant isolément

Référence : React Thinking in React - <https://react.dev/learn/thinking-in-react>

6 Bonnes Pratiques

6.1 Organisation des Fichiers

Structure recommandée :

```
src/
  components/          # Tous les composants
    Header.jsx
    Header.css
    Card.jsx
    Card.css
    Footer.jsx
    Footer.css
  assets/              # Images, fonts, etc.
    logo.svg
  styles/              # Styles globaux (optionnel)
    global.css
  App.jsx              # Composant racine
  App.css
  main.jsx             # Point d'entrée
  index.css            # Styles de base
```

Source : React File Structure Best Practices - <https://www.robinwieruch.de/react-folder-structure>

6.2 Conventions de Nommage

1. **Composants** : PascalCase (Header, CardProduit, UserProfile)
2. **Fichiers composants** : Même nom que le composant (.jsx)
3. **Fichiers CSS** : Même nom que le composant (.css)
4. **Dossiers** : camelCase ou kebab-case
5. **Variables/fonctions** : camelCase

6.3 Commentaires en JSX

```
1 function App() {
2   return (
3     <div>
4       {/* Commentaire JSX - utilise {} */}
5       <h1>Titre</h1>
6
7       {/*
8         Commentaire multi-lignes
9         en JSX
10      */}
11       <p>Paragraphe</p>
12     </div>
13   );
14 }
```

Référence : GeeksforGeeks React Comments - <https://www.geeksforgeeks.org/how-to-write-comments-in-react/>

7 Debugging et DevTools

7.1 Console du Navigateur

Outils de développement :

- **Chrome** : F12 ou Clic droit → Inspecter
- **Firefox** : F12 ou Clic droit → Examiner l'élément
- **Edge** : F12

7.2 React Developer Tools

React DevTools

Extension de navigateur indispensable pour développer en React. Permet d'inspecter la hiérarchie de composants, voir les props et le state.

Installation :

- **Chrome** : <https://chrome.google.com/webstore/detail/react-developer-tools>
- **Firefox** : <https://addons.mozilla.org/fr/firefox/addon/react-devtools/>

Source : React DevTools Documentation - <https://react.dev/learn/react-developer-tools>

7.3 console.log() en React

```
1 function MyComponent() {  
2   const data = "Test";  
3  
4   console.log("Composant rendu");  
5   console.log("Data:", data);  
6  
7   return <div>{data}</div>;  
8 }
```

Attention : Les `console.log()` s'affichent dans la console du navigateur, pas dans le terminal !

8 Récapitulatif

8.1 Concepts Clés Maîtrisés

1. Installation environnement React avec Vite
2. Structure d'un projet React
3. Création de composants fonctionnels
4. Règles du JSX
5. Export/Import de composants
6. Composition de composants

8.2 Prochaines Étapes (Séance 4)

- Props : Passer des données aux composants
- Rendre les composants dynamiques
- Communication parent $\rightarrow$ enfant
- PropTypes et validation

9 Ressources Professionnelles

9.1 Documentation Officielle

1. **React Documentation (2024)**
<https://react.dev>
Documentation officielle complète et moderne
2. **Vite Documentation**
<https://vitejs.dev>
Guide complet de Vite
3. **Node.js Documentation**
<https://nodejs.org/docs>
Documentation Node.js et npm

9.2 Tutoriels et Guides

1. **W3Schools React Tutorial**
<https://www.w3schools.com/react/>
Tutoriel pratique avec exemples interactifs
2. **GeeksforGeeks React**
<https://www.geeksforgeeks.org/react/>
Articles détaillés sur tous les concepts React
3. **MDN Web Docs - JavaScript**
<https://developer.mozilla.org/fr/docs/Web/JavaScript>
Référence JavaScript complète
4. **freeCodeCamp React Course**
<https://www.freecodecamp.org/learn/front-end-development-libraries/>
Cours gratuit complet

9.3 Vidéos (Recommandées)

1. **React Tutorial for Beginners**
Programming with Mosh
<https://www.youtube.com/watch?v=SqcYOGlETPk>
2. **React Course - Beginner's Tutorial**
freeCodeCamp
<https://www.youtube.com/watch?v=bMknfKXIFA8>

9.4 Pratique Interactive

1. **React Official Tutorial**
<https://react.dev/learn/tutorial-tic-tac-toe>
Construire un jeu Tic-Tac-Toe
2. **Scrimba React Course**
<https://scrimba.com/learn/learnreact>
Cours interactif gratuit

3. CodeSandbox

<https://codesandbox.io/s/react>

Éditeur en ligne pour React

9.5 Communauté et Support

1. Stack Overflow - React Tag

<https://stackoverflow.com/questions/tagged/reactjs>

Questions et réponses

2. Reddit - r/reactjs

<https://www.reddit.com/r/reactjs/>

Communauté active

3. Discord - Reactiflux

<https://www.reactiflux.com/>

Chat en temps réel avec des développeurs React

9.6 Livres Recommandés

1. Learning React (2nd Edition)

Alex Banks & Eve Porcello

O'Reilly Media

2. React - The Complete Guide

Maximilian Schwarzmüller

Udemy (cours en ligne)

Prêt pour la Pratique !

Maintenant que vous maîtrisez les fondamentaux,
passons au TP guidé pour créer vos premiers composants !

Bonne pratique !